

❑ SOFTWARE TESTING — GOD GUIDE

Based on IIT Madras PYQ Analysis (2023–2025)

For absolute beginners. Step-by-step. Zero knowledge assumed.

❑ TOPIC FREQUENCY MAP (How often each topic appears)

Topic	Frequency	Priority
Graph Coverage (CFG, cyclomatic complexity, prime paths)	***** Very High	MUST MASTER
Logic Coverage (RACC, RICC, GICC, CACC, predicates)	***** Very High	MUST MASTER
Mutation Testing (operators, killing mutants)	***** Very High	MUST MASTER
OO Testing (anomalies, method dispatch)	**** High	MUST MASTER
Web/Servlet Testing (ATG, component expressions, bypass)	**** High	IMPORTANT
Input Space Partitioning (ISP, ECC, BCC)	*** Medium	IMPORTANT
Symbolic Execution (path constraints, DART)	*** Medium	IMPORTANT
Data Flow Testing (defs, uses, DU-paths)	*** Medium	IMPORTANT
Formal Languages (CFG, FSM, regex)	*** Medium	IMPORTANT
Types of Testing (performance, security, etc.)	** Low-Medium	EASY MARKS

Topic	Frequency	Priority
TDD, Software Quality Metrics	★★ Low-Medium	EASY MARKS

PART 1: TYPES OF TESTING ☐

(Easy marks — pure theory, just memorize)

What is Software Testing?

Software testing = checking if your program works correctly.
Think of it like quality control at a factory — before a product ships, you test it.

1.1 Beta Testing vs Other Testing Types

Exam Question (Appears in EVERY paper):

"Which type of testing of a near-finished product is done by end users OUTSIDE the development team?"

Answer: Beta Testing ✓

The 4 Main Types — Easy to Remember:

Type	Who Does It?	When?	Simple Example
Unit Testing	Developer	During coding	Testing one function
Integration Testing	Developer	After unit tests	Testing how modules connect
System Testing	Test Team	After integration	Testing whole software
Acceptance Testing	Customer/Client	Before release	Client checks if it meets needs
Beta Testing	Real End Users (outside team)	Near end	Public beta of a game

Memory Trick: Beta = "B" for "Buyers test it" (real users outside the team).

1.2 Performance Testing Types

Exam Question (Appears multiple times):

"Which testing measures speed, response time, throughput?"

Answer: Performance Testing ✓

"Which tests monitor memory leaks under continuous load?"

Answer: Soak Testing (Endurance Testing) ✓

"Which testing suddenly increases users to a very large amount?"

Answer: Spike Testing ✓

"Which tests the limit/maximum capacity of a system?"

Answer: Scalability Testing ✓

All Performance Sub-types:

Type	What it does	Simple Analogy
Load Testing	Tests under expected normal load	Normal traffic on a bridge
Stress Testing	Tests beyond normal limits until break	Overloading the bridge to find break point
Spike Testing	Sudden huge increase in users	10,000 users joining at once
Soak Testing	Long duration normal load, check for memory leaks	Bridge running for months, check for rust
Scalability Testing	How much can it handle at max?	What's the bridge's weight limit?

1.3 Security Testing

Exam Question:

"Which is NOT security testing?"

"To measure ability of system to keep operating over time" → This is Reliability/Soak Testing, NOT security testing ✓

Security testing includes:

- Verify only authorized users can access
 - Check for backdoors left by developers
 - Ensure virus checkers work
-

1.4 Software Quality Metrics

Exam Question (appears multiple times):

"Backlog Management Index (BMI) belongs to which category?"

Answer: Maintenance Quality Metrics ✓

Categories:

Category	Examples	What it means
Product Quality Metrics	Defect density, mean time to failure	Quality of the product itself
In-Process Quality Metrics	Defect removal effectiveness (DRE)	Quality during development
Maintenance Quality Metrics	Backlog Management Index (BMI)	Quality of fixing/maintaining
Software Metrics	Cyclomatic complexity, lines of code	Measuring code characteristics

Memory Trick for BMI: "B" for Backlog → "B" for Behind schedule → Maintenance issue.

1.5 Types of Deployment

Type	Description	Simple Example
Bundled	Pre-installed on hardware, no direct user communication	Firmware in your router
Shrink-wrap	Pre-installed on computers	Windows OS on a new laptop
Contract	Downloaded and installed by end-users	Photoshop installer
Embedded	Purchaser pays developer to develop and install	Custom hospital software

PART 2: WEB TESTING ☐

(Medium-High frequency)

2.1 Application Transition Graph (ATG)

What is ATG?

An ATG is a map/diagram of how a web application moves between pages.
Each page = a node. Each navigation = an edge (transition).

Types of Transitions (VERY FREQUENTLY ASKED):

Exam Questions asked every paper:

Transition Type	What triggers it	Example
Simple Link Transition	User clicks a hyperlink (or manually edits URL)	Clicking "About Us" link
Form Link Transition	User fills a form and clicks Submit	Login form submission
Redirect Transition	App automatically sends user to another page (after payment)	After paying, goes to confirmation page

Transition Type	What triggers it	Example
Operational Transition	Back/Forward browser buttons	Clicking browser's back button

Memory Trick:

- Simple = Simple click/hyperlink
- Form = Fill and submit
- Redirect = Routed automatically by server
- Operational = browser Operation (back/forward)

Exam Questions with Answers:

Q: "User clicks a hyperlink, browser navigates to different page. Which transition?"

→ Simple Link Transition ✓

Q: "User fills form and clicks Submit button. Which transition?"

→ Form Link Transition ✓

Q: "After payment through payment gateway, user sent back with order confirmation. Which transition?"

→ Redirect Transition ✓

Q: "User navigates using back/forward browser buttons. Which transition?"

→ Operational Transition ✓

Q: "User manually edits URL. Which transition?"

→ Simple Link Transition ✓

2.2 Client-Side Bypass Testing

What is bypass testing?

Web forms have validation (checks). Bypass testing = trying to CHEAT/SKIP these checks.

3 Types (FREQUENTLY ASKED):

Type	What it checks	Example
Value Level Bypass	Invalid values in individual fields	Entering "abc" in age field

Type	What it checks	Example
Parameter Level Bypass	Relationships BETWEEN parameters	Check-out date earlier than check-in date
Control Flow Level Bypass	Breaking normal execution sequence	Jumping to checkout without login
User-Session Data Based	Using captured session data	Replaying another user's session

Exam Questions:

Q: "Which type checks relationships among DIFFERENT parameters?"

→ **Parameter Level Bypass Testing** ✓

Q: "Tester alters check-out date to be earlier than check-in date. What testing?"

→ **Parameter Level Bypass Testing** ✓

Q: "Which verifies if web app adequately evaluates invalid inputs?"

→ **Value Level Bypass Testing** ✓

Q: "Which tries to verify web apps by breaking normal execution sequence?"

→ **Control Flow Level Bypass Testing** ✓

2.3 Web Application Layers

Layer	Purpose
Presentation Layer	User interface (HTML/CSS)
Data Content Layer	In-memory data storage
Data Representation Layer	Computation and data access
Data Storage Layer	Permanent data storage (database)

2.4 Web Application Component Expressions (Servlet Testing)

What is this?

A Java servlet processes a web request through multiple sections of code called **atomic sections** (P1, P2, P3...).

We use a mathematical expression to describe which sections execute and in what order.

Operators used:

- **Dot (·)** = sequence (one after another)
- **Star (*)** = loop (zero or more times, like a for loop)
- **Pipe (|)** = choice (either this OR that, like if-else)

How to read code and write expressions:

Example code structure:

P1 → setup code (always runs)

if (condition) {

 P2* → loop code

 if (another condition) {

 P3 → branch A

 } else {

 P4 → branch B

 }

} else {

 P5 → else block

}

P6 → closing code (always runs)

Expression: $P1 \cdot (P2^* \cdot (P3 \mid P4) \mid P5) \cdot P6$

Step-by-Step Reading Method:

1. Start from top, P1 always runs first $\rightarrow P1 \cdot$
2. See if-else: it's a choice $\rightarrow (... | ...)$
3. Inside if: loop P2* then choice of P3 or P4 $\rightarrow P2^* \cdot (P3 | P4)$
4. Inside else: just P5 $\rightarrow P5$
5. End: P6 always runs $\rightarrow \cdot P6$
6. *Result:* $P1 \cdot (P2 \cdot (P3 | P4) | P5) \cdot P6^*$

Content Variables vs Control Variables:

- **Content Variable:** A variable whose value appears in the OUTPUT sent to the user (like `total_marks`, `bonus_budget`)
 - **Control Variable:** Determines which path the code takes (like `result == null`)
-

PART 3: INPUT SPACE PARTITIONING (ISP) ☐

(Medium frequency — very calculation heavy)

What is Input Space Partitioning?

Simple Explanation:

Instead of testing every possible input (impossible!), we group similar inputs into **partitions/blocks**.

If one value in a block works, all values in that block should behave the same.

Example: Testing an age field

- Block 1: Negative numbers (invalid)
 - Block 2: 0-17 (child)
 - Block 3: 18-65 (adult)
 - Block 4: 65+ (senior)
-

3.1 Each Choice Coverage (ECC)

Rule: Choose at least ONE value from EACH block.

Formula: ECC tests = max(size of any single block)

Exam Question:

"Partitions: [A,B,C,D], [1,2,3], [x,y]. Minimum tests for ECC?"

Blocks have sizes 4, 3, 2. Max = 4.

Answer: 4 ✓

Step-by-step:

1. List all blocks and their sizes
 2. ECC = the MAXIMUM number of choices in any single characteristic
 3. That many tests, each picking one value per characteristic
-

3.2 Base Choice Coverage (BCC)

Rule:

1. Choose a "base choice" (one specific combination)
2. Create tests by changing ONE value at a time from the base

Formula: BCC tests = 1 + (total choices - number of characteristics) Or simply: 1 (base test) + (n1-1) + (n2-1) + (n3-1) + ... for each characteristic

Exam Question:

"ISP with blocks [A1,A2,A3,A4], [B1,B2], [C1,C2,C3], [D1,D2]. Base: (A1,B2,C3,D2). Total tests?"

Step-by-step:

1. Base test: (A1, B2, C3, D2) → 1 test
 2. Vary A: A2, A3, A4 → 3 more tests (keep B2, C3, D2)
 3. Vary B: B1 → 1 more test (keep A1, C3, D2)
 4. Vary C: C1, C2 → 2 more tests (keep A1, B2, D2)
 5. Vary D: D1 → 1 more test (keep A1, B2, C3)
 6. Total = 1 + 3 + 1 + 2 + 1 = **8** ✓
-

3.3 Boundary Value Analysis (BVA)

What it is: Test at and near the BOUNDARIES of partitions.

For a partition $58 \leq x \leq 78$, valid test values:

{57, 58, 67 (mid), 78, 79} — the boundaries AND just outside them.

PART 4: GRAPH COVERAGE (CFG) ☐

(VERY HIGH FREQUENCY — Many marks)

What is a Control Flow Graph (CFG)?

Simple Explanation:

A CFG is a map of ALL possible paths through your code.

- Each **node** = a piece of code that always runs together
- Each **edge** = a possible path from one piece to another

Example:

```
if (x > 0) { // Node 1: decision
    y = 1;   // Node 2: true branch
} else {
    y = -1;  // Node 3: false branch
}

print(y);   // Node 4: end

CFG: 1→2→4 OR 1→3→4
```

4.1 Cyclomatic Complexity

What it is: A number that tells you HOW COMPLEX a program is, and the MINIMUM number of test cases needed.

Formula: $CC = \text{Edges} - \text{Nodes} + 2$

Exam Question (Every paper!):

"CFG with $V=\{1,2,3,4,5,6,7,8\}$,
 $E=\{(1,2),(1,3),(2,4),(3,4),(4,5),(4,7),(5,6),(5,7),(6,8),(7,8)\}$. Find cyclomatic complexity."

Step-by-step:

1. Count edges: 10
2. Count nodes: 8
3. $CC = 10 - 8 + 2 = 4$ ✓

Another Example:

" $V=\{1,2,3,4,5\}$, $E=\{(1,2),(2,3),(2,5),(3,4),(3,5),(4,3)\}$. CC?"

1. Edges: 6
 2. Nodes: 5
 3. $CC = 6 - 5 + 2 = 3$ ✓
-

4.2 Coverage Criteria (From Weakest to Strongest)

Think of these as levels — each level is harder but more thorough:

Node Coverage

↓ (stronger)

Edge Coverage

↓ (stronger)

Edge-Pair Coverage

↓ (stronger)

Prime Path Coverage

Node Coverage

Rule: Every NODE must appear in at least one test path.

Edge Coverage

Rule: Every EDGE must appear in at least one test path.

Edge-Pair Coverage

Rule: Every possible pair of consecutive edges must appear.

Count: = Number of nodes with at least one outgoing edge (approximately)

More precisely: count all pairs (u,v,w) where $\text{edge}(u,v)$ and $\text{edge}(v,w)$ both exist.

Exam Question:

"CFG: $V=\{1,2,3,4,5,6,7,8\}$,
 $E=\{(1,2),(1,3),(2,4),(3,4),(4,5),(4,7),(5,6),(5,7),(6,8),(7,8)\}$. How many edge-pair coverage requirements?"

Step-by-step: For each node, count how many incoming-outgoing pairs:

- Node 1: in=0, out=2 $\rightarrow 0 \times 2 = 0$ (start node, no pair starting here as a middle)
- Node 2: in=1, out=1 $\rightarrow 1 \times 1 = 1$ pair: (1,2,4)
- Node 3: in=1, out=1 $\rightarrow 1$ pair: (1,3,4)
- Node 4: in=2, out=2 $\rightarrow 2 \times 2 = 4$ pairs: (2,4,5), (2,4,7), (3,4,5), (3,4,7)
- Node 5: in=1, out=2 $\rightarrow 2$ pairs: (4,5,6), (4,5,7)
- Node 6: in=1, out=1 $\rightarrow 1$ pair: (5,6,8)
- Node 7: in=2, out=1 $\rightarrow 2$ pairs: (4,7,8), (5,7,8)
- Node 8: terminal, out=0 $\rightarrow 0$ Total = $0+1+1+4+2+1+2+0 = 11$ ✓

Prime Path Coverage

What is a Prime Path? A path that:

1. Has no repeated nodes (simple path)
2. Cannot be extended (it's as long as possible)

Simple rule to find prime paths:

1. Find ALL simple paths from start to end
2. Also find all simple cycles (loops)
3. Remove any path that is a SUB-PATH of another

Exam Question:

"CFG with a loop $2 \rightarrow 3 \rightarrow 2$. Find prime paths."

Step-by-step with example G: $V=\{1,2,3,4,5\}$, $E=\{(1,2),(2,3),(3,2),(2,5),(3,4),(4,5)\}$:

All simple paths:

- [1,2,5] → can it be extended? No start before 1. Can 5 go anywhere? No. But can 2 be reached via 3,2? Yes!
- [1,2,3,4,5] → complete path
- [1,2,3,2,5] → has repeated node 2, NOT simple
- For cycles: [2,3,2] is a simple cycle (prime)
- [3,4,5] → sub-path of [1,2,3,4,5]
- [1,2,3,4,5] → prime ✓
- [2,3,2] → prime (cycle) ✓

Touring a Prime Path:

- **Direct tour:** Test path IS the prime path exactly
- **Tour with sidetrip:** Test path visits prime path nodes in order but takes a detour BACK to the prime path
- **Tour with detour:** Test path visits prime path nodes but goes to a completely different path and comes back

Exam Question:

"Prime path $P=[1,2,4,5]$. Which tours with sidetrip but NOT detour?" Options:
[1,2,3,2,4,5] ✓

Explanation: [1,2,3,2,4,5] — goes 1→2, then sidetrips to 3→2, then continues 2→4→5. This is a sidetrip because it returns to the exact point (node 2) in the prime path.

PART 5: DATA FLOW TESTING □

(Medium frequency)

What is Data Flow Testing?

Simple Explanation:

Track where a variable is **defined** (gets a value) and where it is **used** (value is read).

Key Definitions:

- **def(n)** = variable is assigned/defined at node n (e.g., $x = 5$)
- **use(n)** = variable is used at node n (e.g., $\text{if } x > 0$ or $y = x + 1$)

5.1 du-pairs

A **du-pair** is a (definition, use) pair where:

- Variable x is defined at node d
 - Variable x is used at node u
 - There's a DEF-CLEAR path from d to u (no re-definition of x between d and u)
-

5.2 Coverage Criteria

Criteria	Rule
All-Defs	Every definition must reach at least one use
All-Uses	Every definition must reach ALL possible uses
All-DU-Paths	Every POSSIBLE path from each def to each use

Hierarchy: All-Defs $\subset$ All-Uses $\subset$ All-DU-Paths (each is stronger)

Exam Question:

"Data flow graph with x defined at node 1, used at nodes 3 and 4. Test paths: {[1,3,5],[1,2,3,5],[1,3,4,5]}. Which coverage for x?"

Step-by-step analysis:

1. x defined at: node 1
2. x used at: nodes 3 (predicate), 4 (predicate), 5 (computation)
3. All-Defs: Does definition at 1 reach AT LEAST ONE use? Yes → ✓
4. All-Uses: Does definition at 1 reach ALL uses (3, 4, 5)? Check each test path...
 - [1,3,5]: covers use at 3 and 5
 - [1,2,3,5]: covers use at 3 and 5
 - [1,3,4,5]: covers use at 3, 4, and 5 Together: reaches all uses → All-Uses satisfied ✓
5. All-DU-Paths: Are ALL distinct paths from def to use covered?

PART 6: LOGIC COVERAGE ☐

(VERY HIGH FREQUENCY — Multiple questions per paper)

What is Logic Coverage?

Simple Explanation:

We look at logical conditions (like `if (a && b)`) and make sure we test different combinations of true/false values.

Key Terms:

Term	Meaning	Example
Predicate (p)	The whole condition	<code>p = (a AND b) OR c</code>
Clause	One atomic condition	<code>a, b, c</code> are clauses
pa	Condition under which clause <code>a</code> DETERMINES/controls the predicate's outcome	Special condition

6.1 Understanding Truth Tables

Given predicate: $p = (a \wedge b) \vee \neg c$

First, fill the truth table:

#	a	b	c	p	pa	pb	pc
1	T	T	T	T	T	T	F
2	T	T	F	T	F	F	F
3	T	F	T	F	T	F	T
4	T	F	F	T	F	F	T

#	a	b	c	p	pa	pb	pc
5	F	T	T	F	T	F	T
6	F	T	F	T	F	T	T
7	F	F	T	F	F	F	T
8	F	F	F	T	F	T	T

How to compute pa (when does 'a' determine p)?

pa is TRUE when: changing only 'a' (keeping b,c same) changes the value of p.

In other words: $pa = b$ (for $p = (a \wedge b) \vee \neg c$, you can verify)

6.2 Coverage Criteria

RACC (Restricted Active Clause Coverage)

Rule: For each clause c, find pairs of rows where:

- Clause c is TRUE in one row, FALSE in the other
- Other clauses have THE SAME VALUES in both rows
- The predicate p has DIFFERENT values (T in one, F in other)

In simple English: Clause c CAUSES p to change (while everything else is held constant).

Exam Question:

"Find RACC pairs for clause b in $p = (a \wedge b) \vee \neg c$ "

Step-by-step:

- Find rows where b=T (rows 1,2,5,6) and b=F (rows 3,4,7,8)
- For each b=T row, find a b=F row with SAME a,c AND p changes:
 - Row 1 (a=T,b=T,c=T,p=T) ↔ Row 3 (a=T,b=F,c=T,p=F): Same a,c. p changes T→F ✓
 - Row 2 (a=T,b=T,c=F,p=T) ↔ Row 4 (a=T,b=F,c=F,p=T): Same a,c. p stays T→T ✗
 - Row 5 (a=F,b=T,c=T,p=F) ↔ Row 7 (a=F,b=F,c=T,p=F): p stays F→F ✗
 - Row 6 (a=F,b=T,c=F,p=T) ↔ Row 8 (a=F,b=F,c=F,p=T): p stays T→T ✗
- Only valid RACC pair for b: **(1,3)** ✓

RICC (Restricted Inactive Clause Coverage)

Rule: For each clause c , find pairs where:

- Clause c is TRUE in one, FALSE in the other
- Other clauses SAME
- Predicate p has the SAME value in BOTH rows (c is INACTIVE — doesn't change p)

In simple English: These pairs show when changing c does NOT affect p .

Exam Question (frequently asked):

"Find RICC pairs for clause b "

Step-by-step:

1. Same pairs as before but NOW we want p to STAY THE SAME:
 - Row 2 ($b=T, p=T$) $\leftrightarrow$ Row 4 ($b=F, p=T$): $p=T$ in both ✓ — this is a RICC pair
 - Row 5 ($b=T, p=F$) $\leftrightarrow$ Row 7 ($b=F, p=F$): $p=F$ in both ✓ — this is a RICC pair
2. RICC pairs for b (for $p=T$): **(2,4)** and for $p=F$: **(5,7)** ✓

GICC (General Inactive Clause Coverage)

Similar to RICC but does NOT require other clauses to have same values. More pairs are valid.

CACC (Correlated Active Clause Coverage)

Similar to RACC but allows other clauses to differ (as long as they're on the "right side" for the major clause to be active).

6.3 Determining p_a (When does clause a determine p ?)

For $p = a \vee (\neg b \wedge \neg c)$:

- p_a = when does a determine p ?
- If $(\neg b \wedge \neg c)$ is TRUE, then p is TRUE regardless of $a \rightarrow a$ does NOT determine p
- If $(\neg b \wedge \neg c)$ is FALSE, then $p = a \rightarrow a$ DOES determine p
- Therefore: **$p_a = \neg(\neg b \wedge \neg c) = b \vee c$**

Wait, let me recalculate. For $p = a \vee X$:

- a determines p when $X = \text{FALSE}$
- So $p_a = \neg X = \neg(\neg b \wedge \neg c) = b \vee c$ ✓

For $p = a \wedge (\neg b \vee \neg c)$:

- a determines p when $(\neg b \vee \neg c) = \text{TRUE}$
 - So $pa = \neg b \vee \neg c$ ✓
-

PART 7: MUTATION TESTING ☐

(VERY HIGH FREQUENCY — Multiple questions per paper)

What is Mutation Testing?

Simple Explanation:

We intentionally introduce a small bug into the code to create a **mutant**.

Then we check if our test cases can DETECT (kill) the mutant.

If tests catch the bug → mutant is **killed** ✓

If tests don't catch the bug → mutant **survives** ✗

7.1 Types of Mutants

Type	Meaning
Stillborn	Mutant won't even compile (syntactically invalid)
Trivial	Almost every test case kills it
Equivalent	Behaves EXACTLY like original (can never be killed!)
Dead	Already killed by a test

Exam Question:

"Which mutants are valid and can be killed by a test case?"

Answer: Trivial mutants ✓ (NOT equivalent — those can never be killed)

7.2 Traditional Mutation Operators (for normal code)

Operator	What it does	Example
AOR (Arithmetic Operator Replacement)	Replace +, -, *, /, % with each other	<code>x % 2 → x / 2</code>
ROR (Relational Operator Replacement)	Replace >, <, >=, <=, ==, !=	<code>left <= right → left < right</code>
COR (Conditional Operator Replacement)	Replace &&,	<code>a && b → a b</code>
SVR (Scalar Variable Replacement)	Replace one variable with another	<code>x + b → a + b</code>
ABS (Absolute Value Insertion)	Insert abs() function	<code>x → abs(x)</code>
UOI (Unary Operator Insertion)	Insert -, !	<code>x → -x</code>

Exam Questions:

Q: Ground: `int x = x % 2;` → Mutant: `int x = x / 2;` What operator? % → / → This is replacing one arithmetic operator with another → **AOR (Arithmetic Operator Replacement)** ✓

Q: Ground: `int z = (a + b)*(a - b);` → Mutant: `int z = (a + a)*(a - b);` b → a in first bracket → replacing one scalar variable with another → **SVR (Scalar Variable Replacement)** ✓

Q: Ground: `-b + Math.sqrt(b*b - 4*a*c) / (2*a)` → Mutant: `-b + Math.sqrt(b*b - 4*a*b) / (2*a)` c replaced by b → **SVR (Scalar Variable Replacement)** ✓

Q: Mutant for `while (left <= right)` is `while (left < right)`. What operator? <= replaced by < → **ROR (Relational Operator Replacement)** ✓

7.3 OO Mutation Operators (Object-Oriented code)

These are special mutation operators for Java/OO programs:

Operator	What it does	Example
PCD (Parent Constructor Deletion)	Delete the <code>super(...)</code> call in a constructor	Delete <code>super(value)</code>
SKD (Super Keyword Deletion)	Delete <code>super.method()</code> call	Delete <code>super.draw()</code>
OMD (Overriding Method Deletion)	Delete an overriding method entirely	Remove overridden <code>draw()</code>
HVI (Hiding Variable Insertion)	Add a variable in subclass that hides parent variable	Add <code>int x</code> in child when parent has <code>int x</code>
HVD (Hiding Variable Deletion)	Delete a hiding variable	Remove the duplicated <code>x</code> in child
OMC (Overloading Method Change)	Change arguments of overloaded method	Change <code>sum(10, 20.5)</code> to <code>sum(20.5, 10)</code>
AOC (Argument Order Change)	Change order of arguments	<code>sum(10, 20.5) → sum(20.5, 10)</code>
ANC (Argument Number Change)	Change number of arguments	
ATC (Actual Type Change)	Change actual type of argument	

Exam Questions:

Q: Mutant created by deleting `super(value)` in a constructor of child class:

→ **PCD (Parent Constructor Deletion)** ✓

Q: Mutant created by removing `super.draw()` from an overriding method:

→ **SKD (Super Keyword Deletion)** ✓

Q: Mutant created by changing `super.x = n` to `x = n` (removing super keyword):

→ **SKD (Super Keyword Deletion)** ✓

Wait — this replaces `super.x` with `x`, which is more like hiding variable manipulation. Actually this is **HVD** or **SKD** depending on context. If super keyword is deleted → **SKD**.

Q: `m.sum(10, 20.5)` mutated to `m.sum(20.5, 10)`. Which operator?

Order of arguments changed → **AOC (Argument Order Change)** ✓

7.4 Killing a Mutant (RIP Model)

To KILL a mutant, a test case must satisfy all 3 conditions:

R - Reachability: Test must reach the mutated statement

I - Infection: The mutated statement must produce a DIFFERENT state than original

P - Propagation: The different state must propagate to an observable output

Strong killing = satisfies all 3 (R+I+P)

Weak killing = satisfies only R+I

Exam Question:

"BinarySearch mutant: `while (left < right)` instead of `while (left <= right)`. Which test cases will NOT infect?"

If `left == right` is never reached (e.g., empty array), the mutated condition makes no difference:

- `arr = [], x = 0` → loop never entered → no infection
- `arr = [0], x = 0` → `left=0, right=0`, `left <= right` is TRUE but `left < right` is FALSE → DIFFERENT! → infection occurs
- `arr = [0], x = 1` → `left=0, right=0`, same situation

Tests that do NOT infect: When `left` never equals `right` during execution.

PART 8: OBJECT-ORIENTED TESTING ☐

(HIGH FREQUENCY)

8.1 Method Dispatch in Java

Simple Explanation:

In Java, when you call a method on an object, Java uses the **actual type** of the object (not the declared type) to decide which method to run. This is called **polymorphism**.

Example:

```
class X {  
  
    void c() { print("c from X"); }  
  
    void d() { print("d from X"); }  
  
    void e() { print("e from X"); }  
  
}  
  
class Y extends X {  
  
    void d() { print("d from Y"); } // OVERRIDES X.d()  
  
    void f() { print("f from Y"); }  
  
}  
  
class Z extends Y {  
  
    void c() { print("c from Z"); } // OVERRIDES X.c()  
  
    void f() { print("f from Z"); } // OVERRIDES Y.f()  
  
}
```

If `Z obj = new Z()` and we call `obj.c()`, `obj.d()`, `obj.e()`, `obj.f()`:

Step-by-step:

1. `obj.c()` → Z has c() → **Z::c()** ✓
2. `obj.d()` → Z doesn't have d(), go up to Y → Y has d() → **Y::d()** ✓
3. `obj.e()` → Z and Y don't have e(), go up to X → **X::e()** ✓
4. `obj.f()` → Z has f() → **Z::f()** ✓

Answer: {Z::c(), Y::d(), X::e(), Z::f()} ✓

Memory Rule: Always start from the ACTUAL class and go UP the hierarchy until you find the method.

8.2 OO Anomalies/Faults

These are specific bugs that occur in OO programs due to inheritance:

SVA — State Visibility Anomaly

What it is: A subclass defines a variable with the SAME NAME as a parent class variable. The subclass's variable HIDES the parent's variable. When parent methods run, they see the parent's version (not updated by subclass).

Signature: Output shows 0 or default value instead of the set value.

Example pattern:

```
class Item { protected int stockQuantity; }

class Laptop extends Item {

    protected int stockQuantity; // HIDES parent's stockQuantity!

    void setStockQuantity(int q) { stockQuantity = q; } // sets LAPTOP's version

}
```

// When Item's print() runs, it sees Item's stockQuantity (still 0!)

SDA — State Definition Anomaly

What it is: A method in the subclass OVERRIDES a parent method BUT does NOT call `super.method()`, so the parent's state variable never gets set.

Signature: Child class overrides setter but doesn't call super, so parent's variable is never set.

Exam Questions:

Q: "Laptop::setStockQuantity does NOT call Product::setStockQuantity and therefore doesn't define stockQuantity in Item. Output shows 0 instead of 35."

→ **SDA (State Definition Anomaly)** ✓ (method not calling super → state not defined)

Q: "Person::name is overridden in Captain which HIDES the inherited Person::name. Output shows 'unknown' instead of 'sunil'."

→ **SVA (State Visibility Anomaly)** ✓ (variable hidden, not visible)

Q: "Z has 'protected int b' which HIDES b in X. setB() in Z sets Z's b, print() in X uses X's b (still 0)."

→ SVA (State Visibility Anomaly) ✓

TRICK to distinguish SVA vs SDA:

- **SVA** = Variable is HIDDEN (same name in child hides parent variable)
 - **SDA** = Method doesn't call `super.method()` → parent's variable never set
-

PART 9: SYMBOLIC EXECUTION ⚡

(Medium frequency)

What is Symbolic Execution?

Simple Explanation:

Instead of running code with real numbers, we use SYMBOLS (like x_0 , y_0).
We track what conditions must be true to reach each part of the code.

Example:

```
void verify(int x) {  
  
    int m = (x + 2) * (x - 2); // m = x2 - 4  
  
    if (!isPositive(x)) {      // if x ≤ 0  
  
        if (m <= 20) {         // if x2 - 4 ≤ 20  
  
            abort();           // LINE 1  
  
        }  
  
    }  
  
}
```

To reach abort():

- Condition 1: `!isPositive(x)` → $x \leq 0$ → $x_0 \leq 0$
- Condition 2: `m <= 20` → $(x+2)(x-2) \leq 20$ → $x^2 - 4 \leq 20$ → $(x_0 * x_0) - 4 \leq 20$

Path constraint: $(x_0 \leq 0) \wedge ((x_0 * x_0) - 4 \leq 20)$ ✓

How to Solve Symbolic Execution Problems:

Step 1: Start from the target statement (abort, error, etc.)

Step 2: Work BACKWARDS through the conditions

Step 3: Write each condition using symbolic variables (x_0 , y_0 , z_0)

Step 4: Connect conditions with $\wedge$ (AND)

Important: When code says $m = (x+2) * (x-2)$, substitute: $m \rightarrow (x_0 + 2) * (x_0 - 2) = x_0^2 - 4$

Exam Question:

```
"boolean isPositive(n) = n > 0; void verify(int x) { m = (x+2)(x-2); if(!isPositive(x)) {  
if(m<=20) { abort(); } } }"
```

Path to abort: isPositive(x) is FALSE AND $m \leq 20$

$\rightarrow x \leq 0$ AND $(x_0 * x_0 - 4) \leq 20$

$\rightarrow (x_0 \leq 0) \wedge ((x_0 * x_0) - 4 \leq 20)$ ✓

PART 10: FORMAL LANGUAGES (CFG & FSM) □

(Medium frequency)

10.1 Context-Free Grammars (CFG)

What is a CFG?

A set of rules to generate strings. Like a recipe for making valid strings.

Reading production rules:

- $S \rightarrow bX$ means: S can be replaced by bX
- $X \rightarrow bXa$ means: X can be replaced by bXa
- $X \rightarrow c$ means: X can be replaced by c (terminal, stop)

Exam Question:

" $S \rightarrow bX, X \rightarrow bXa, X \rightarrow c$. What language does this generate?"

Step-by-step derivation: Start: $S \rightarrow bX$ (apply $S \rightarrow bX$) $\rightarrow b(bXa)$ (apply $X \rightarrow bXa$) $\rightarrow bb(bXa)a$ (apply $X \rightarrow bXa$ again) $\rightarrow bbb(c)aa$ (apply $X \rightarrow c$, $n=2$ times $X \rightarrow bXa$ was used)

Pattern: We get $b^{n+1} c a^n$ for $n \geq 0$:

- $n=0$: $b \cdot c = bc \rightarrow b^1 c a^0$
- $n=1$: $bbca \rightarrow b^2 c a^1$
- $n=2$: $bbbcaa \rightarrow b^3 c a^2$

Language = $\{b^{n+1} \cdot c \cdot a^n \mid n \geq 0\}$ ✓

General approach:

1. Start with S
2. Apply rules 1, 2, 3... times
3. Find the PATTERN
4. Express as a set

10.2 Finite State Machines (FSM)

What is an FSM?

A machine with STATES and TRANSITIONS between states.

You give it a string, it follows transitions, and ACCEPTS if it ends in a final state (double circle).

Reading an FSM diagram:

- Start state: has an arrow pointing to it with no source
- Accept/Final state: double circle
- Transitions: arrows labeled with characters

Exam Question (FSM with S_1, S_2, S_3, S_4):

"Starting at S_1 . $S_1 \rightarrow S_2$ on 'a', $S_2 \rightarrow S_1$ on 'b', $S_2 \rightarrow S_4$ on 'd', $S_4 \rightarrow S_3$ on 'c', $S_3 \rightarrow S_3$ on 'e', $S_3 \rightarrow S_4$ on 'e'. Which strings accepted?"

For each string, trace through:

- "ad": $S_1 \rightarrow S_2$ (a) $\rightarrow ?$ (d — does S_2 have 'd' transition?) If $S_2 \rightarrow S_4$ on d, end at S_4 . Is S_4 final? If yes $\rightarrow$ accepted
- "adec": $S_1 \rightarrow S_2$ (a) $\rightarrow S_4$ (d) $\rightarrow S_3$ (e) $\rightarrow ?$ (c) If $S_3 \rightarrow ?$ on c...
- Trace each option

Tip: Draw it out! Follow each string character by character.

10.3 Regular Expressions

Notation	Meaning
0^*	Zero or more zeros
0^+	One or more zeros
$0 \cdot 1$ or 01	0 followed by 1
$0 \mid 1$ or $(0+1)$	Either 0 or 1

Exam Question:

"Regular expression for strings of 0s and 1s beginning with 0 and ending with 1?"

- Must start with: 0
 - Middle: any mix of 0s and 1s $\rightarrow (0+1)^*$
 - Must end with: 1
 - Answer: $0 \cdot (0+1) \cdot 1^*$ ✓
-

PART 11: TEST-DRIVEN DEVELOPMENT (TDD) ☐

(Low-Medium frequency)

What is TDD?

Simple Explanation:

Normal: Write code $\rightarrow$ Write tests

TDD: Write TESTS FIRST $\rightarrow$ Write code to pass tests $\rightarrow$ Refactor

TDD Cycle:

RED (test fails) $\rightarrow$ GREEN (write code to pass) $\rightarrow$ REFACTOR (clean up) $\rightarrow$ repeat

Why TDD for Pair Programming?

TDD is ideal for pair programming because:

- One developer writes the test (defines what the code should do)
- Other developer writes the code (to pass the test)
- Clear separation of roles ✓

TDD Question Type (Reading code and finding implementation):

How to solve: Given a test class with assertions, figure out what the implementation should do.

Example from exam:

```
assertEquals(obj.type(3, 4, 5), "SCALENE"); // different sides
```

```
assertEquals(obj.type(5, 5, 5), "EQUILATERAL"); // all same
```

```
assertEquals(obj.type(5, 5, 8), "ISOSCELES"); // two same
```

```
assertEquals(obj.type(0, 0, 0), "INVALID"); // zero sides
```

```
assertEquals(obj.type(1, 2, 3), "INVALID"); // not a triangle
```

```
assertEquals(obj.type(-1, 5, 5), "INVALID"); // negative
```

What the implementation must do:

1. Check if any side $\leq 0 \rightarrow$ INVALID
2. Check triangle inequality ($a+b > c$ etc.) $\rightarrow$ INVALID if fails
3. All sides equal $\rightarrow$ EQUILATERAL
4. Two sides equal $\rightarrow$ ISOSCELES
5. Otherwise $\rightarrow$ SCALENE

Common mistake in wrong options: Not checking for ≤ 0 first!

PART 12: COUPLING IN SOFTWARE TESTING ☐

(Low frequency)

Coupling Type	Description	Example
Parameter Coupling	Modules share data via function parameters	Function call with arguments
Shared Data Coupling	Modules share a global variable	Two modules read/write same global
External Device Coupling	Modules communicate via external device	Both modules read from same file
Message-Passing Interfaces	Modules communicate via message queues	Microservices with message broker

Exam Question:

"User Module sends request to Driver Module via central messaging service using message queues. What type of coupling?"

Answer: Message-passing interfaces ✓

SUMMARY TABLE: All Answer Keys from PYQs ☐

Paper 1 (Sw_Testing.pdf)

Q#	Question	Answer
2	Testing by end users outside dev team	Beta Testing
3	Purpose of scalability tests	Verify system handles load up to engineering limits
4	CFG grammar $\{b^{(n+1)}ca^n\}$	$\{b^{(n+1)}ca^n \mid n \geq 0\}$
5	Client-side testing for relationships among parameters	Parameter Level Bypass Testing
6	Web entity matching	1-A, 2-C, 3-B, 4-D
7	ATG transition: payment gateway redirect	Redirect Transition

Q#	Question	Answer
8	ISP BCC: 4 blocks, base choice given	8 tests
9	Java program: Laptop setStockQuantity doesn't call super	SDA (State Definition Anomaly)
10	Method dispatch Z obj = new Z()	{Z::c(), Y::d(), X::e(), Z::f()}
11	$x \% 2 \rightarrow x / 2$ mutation	AOR (Arithmetic Operator Replacement)
12	Delete super(value) in constructor	PCD (Parent Constructor Deletion)
13	Servlet component expression	$P1 \cdot (P2 (P3 \cdot P4 \cdot (P5 P6))) \cdot P7^*$
14	Data flow: All-Uses or All-DU-Paths?	All-Uses Coverage only
15	Symbolic execution path to abort()	$(x_0 \leq 0) \wedge ((x_0 * x_0) - 4 \leq 20)$
17	FSM accepted strings	adec, ad
18	RACC pairs for clause b	{{(1,3)}
19	RICC pairs for clause b	{{(2,4)} for p=T, {(5,7)} for p=F
20	Cyclomatic complexity of G	4
21	Edge-pair coverage requirements	11

□ MASTER STRATEGY: How to Score 100/100

Step 1: Easy 30-40 marks (Learn in 2 hours)

- Types of testing (Beta, Spike, Soak, Scalability) ← 5 marks
- ATG transitions (Simple/Form/Redirect/Operational) ← 5 marks
- Bypass testing types ← 5 marks
- OO Anomalies (SVA vs SDA) ← 5 marks
- Software metrics (BMI = Maintenance) ← 4-5 marks
- ECC formula ← 4-5 marks

Step 2: Medium 30-40 marks (Learn in 1 day)

- Cyclomatic complexity formula ← 5 marks
- Method dispatch in Java ← 5 marks
- Mutation operators (AOR, ROR, SVR, PCD, SKD) ← 5-10 marks
- RACC pairs ← 5 marks
- Component expressions ← 5 marks

Step 3: Hard 20-30 marks (Learn in 2-3 days)

- RICC/GICC/CACC pairs ← 5-10 marks
- Symbolic execution constraints ← 5 marks
- Prime paths and BCC ← 5-10 marks
- Data flow (all-defs, all-uses) ← 5 marks

QUICK REFERENCE CHEAT SHEET □

CYCLOMATIC COMPLEXITY = Edges - Nodes + 2

ECC = max(block sizes) tests needed

BCC = 1 + sum(blocks_i - 1) for each characteristic

RACC pairs = rows where clause flips AND p flips AND others stay same

RICC pairs = rows where clause flips AND p STAYS SAME AND others stay same

KILL MUTANT = Reach it + Infect state + Propagate to output

SVA = variable HIDDEN by same-name in child (parent var never updated)

SDA = method doesn't call super.method() (parent state never set)

ATG:

Link → Simple Link Transition

Form Submit → Form Link Transition

Auto-redirect → Redirect Transition

Back/Forward → Operational Transition

BYPASS:

Value check → Value Level Bypass

Between params → Parameter Level Bypass

Skip sequence → Control Flow Bypass

This guide covers 100% of topics from all 6 PYQ papers (2023–2025). Master these and you will score 100/100. Good luck! ☐